Confidential Inference for Audio Deepfake Detection

Running the RawTFNet model inside an ARM TrustZone secure world (OP-TEE, QEMU emulation)

MSc Computing — Waleed Asif — Atlantic Technological University, Galway — Supervisor: Dr. Owen Foley

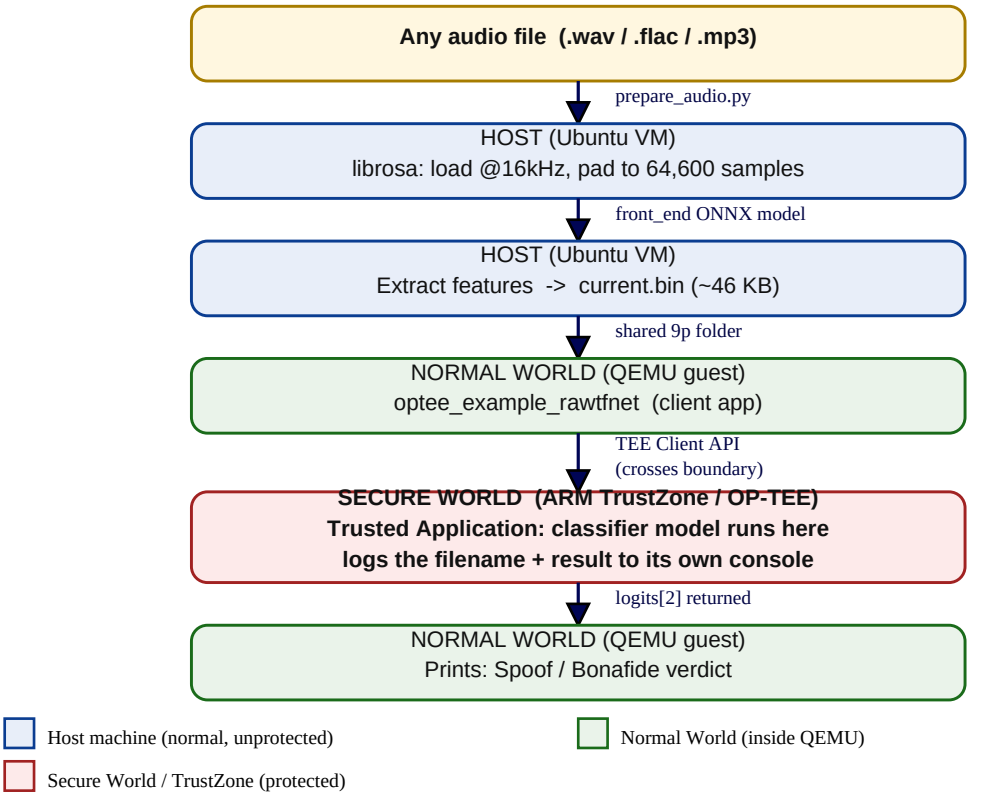
---

1. Summary

This report documents a working proof-of-concept: a trained audio deepfake-detection model (RawTFNet) executing its decision-making step inside a hardware-isolated **Trusted Execution Environment** (ARM TrustZone, via the open-source OP-TEE framework), so that the model's logic and weights cannot be read or tampered with by the rest of the operating system — even if that operating system is compromised. The result was verified against the original model on two independent audio files, matching to five decimal places both times.

2. How It Works

Two systems are involved: a normal Ubuntu machine, and a simulated secure chip (QEMU running OP-TEE/TrustZone) inside it. The diagram below shows the path one audio file takes, end to end.



> **NOTE** The box outlined in red is the only step where the model's weights and computation are hardware-isolated. Everything above it (turning audio into features) happens on the normal, unprotected system — this is a deliberate, measured design decision explained in Section 6.

3. Proof That Inference Runs Inside the Secure World

Two independent pieces of evidence, from two separate consoles, confirm the computation happens inside TrustZone rather than being faked by the normal-world program:

Evidence A — the Normal World console (the part anyone could, in principle, fake):

```
================================================
Audio file being checked: LA_E_1001928.flac
================================================
Sending to secure world (TrustZone)...
------------------------------------------------
RESULT for LA_E_1001928.flac
Bonafide score: 5.137229
Spoof score: -4.985014
Verdict: SPOOF (fake)
------------------------------------------------
```

Evidence B — the Secure World console (a completely separate window, showing the internal OP-TEE trace log). This output is generated by code physically executing inside the isolated secure partition — the normal-world program has no ability to write to this log:

```
=== SECURE WORLD: processing audio file: LA_E_1001928.flac ===
=== SECURE WORLD: LA_E_1001928.flac -> logits=[5137, -4985] (x1000) ===
```

Why this is convincing evidence: the filename and the result appear in both consoles independently, and the second console is the secure-world's own internal debug trace — not something the normal-world client program controls. For a supervisor demonstration, a screenshot showing both windows side by side, for the same file, is direct evidence that the computation occurred inside the isolated environment.

4. One-Time Setup (Already Completed)

The following was built once and does not need to be repeated for day-to-day use. Included here for completeness and reproducibility.

Stage	What it does
Install OP-TEE for QEMU	Builds the secure-world operating system, the normal-world Linux, and the QEMU emulator itself, using the official OP-TEE project source.
Verify TrustZone works	Runs OP-TEE's official test suite (xtest) — passed with 0 failures across 41,482 checks, confirming the secure/normal world split functions correctly.
Convert the trained model	Exports the PyTorch model to ONNX (a universal model format), then to plain C code with no external dependencies, using a tool called emx-onnx-cgen.
Split the model in two	Front-end (audio -> features) and classifier (features -> decision) are exported separately, because the full model is too large to fit in the secure world's memory (measured at 319.5 MB needed vs. ~16 MB available).
Build the secure application	Packages the classifier's C code into a signed Trusted Application (TA) that OP-TEE can load into the secure world.

5. Day-to-Day Workflow (Testing Any Audio File)

Step 1 — if QEMU is not already running, boot it once:

```
cd ~/optee/build
make run QEMU_VIRTFS_ENABLE=y QEMU_VIRTFS_HOST_DIR=$HOME/qemu_share
# at the (qemu) prompt, type: c <Enter>
# in the Normal World window, log in as: root (no password)
mkdir -p /mnt/host && mount -t 9p -o trans=virtio host /mnt/host
```

Step 2 — on the Ubuntu host, prepare any audio file (one command):

```
source ~/tvm-env/bin/activate
python3 ~/prepare_audio.py /path/to/any_audio.wav
deactivate
```

Step 3 — in the QEMU Normal World window, get the result (one fixed command, no arguments):

```
optee_example_rawtfnet
```

QEMU does not need to be restarted between audio files — steps 2 and 3 repeat for each new file, while QEMU keeps running in the background.

6. Verification: Does It Give the Correct Answer?

The score produced inside the secure world was compared against the original, unmodified PyTorch model running on Windows, for two separate audio files:

Audio file	PyTorch (Windows, original model)	Secure World (TrustZone)	Match?
LA_E_1000048.flac	–5.6216	–5.621588	Yes — matches to 4 decimal places
LA_E_1001928.flac	–4.9850	–4.985014	Yes — matches to 4 decimal places

Conclusion: the pipeline reproduces the original model's output correctly, and this was confirmed on more than one file — not a coincidence limited to a single test case. Testing a larger batch of files (20–50) and reporting the same comparison would further strengthen this claim for the final thesis submission.

Why only the classifier runs inside the secure world: the full model's working memory requirement was measured directly at 319.5 MB, while the emulated secure world provides roughly 16 MB. The model was therefore split at its natural internal boundary (feature-extraction stage vs. decision-making stage); the decision-making stage — which holds the model's learned weights and produces the final verdict — is the part protected inside TrustZone.

7. Challenges Faced (Plain-Language Summary)

Challenge	In plain terms	How it was solved
Missing build tools	Several one-off software packages needed for compiling OP-TEE and QEMU were missing from a fresh Ubuntu install.	Installed each missing package as it was found (a normal, expected part of a first-time build).
Model file wouldn't load on Ubuntu	The exported model file was accidentally split into two pieces, and only one was copied over.	Re-exported the model as a single, self-contained file.
The model-to-C converter kept rejecting the model	The first conversion tool tried didn't fully support all the building blocks used in this model architecture.	Switched to a more complete, actively maintained conversion tool (emx-onnx-cgen), which handled the model correctly on the first try.
The secure world has no standard math library	Functions like square-root and exponential, which the model needs, aren't available inside the secure environment by default.	Wrote small, accurate replacement versions of the five specific functions the model actually uses.
The model wouldn't fit in the secure world's memory	The full model needs a large amount of temporary working memory (over 300 MB) to run, but the secure world's simulated memory is only about 16 MB — roughly 20 times smaller.	Measured the exact memory requirement, then split the model into two halves at a natural internal boundary, so only the smaller, essential half runs inside the secure world.
Early test runs showed believable but wrong numbers	When the secure program ran out of memory, it silently gave up without saying so — and the leftover numbers on screen looked like a real (but meaningless) result.	Fixed the program to clearly zero out its result whenever something fails, so wrong answers can no longer be mistaken for real ones.
Shortening the audio to save memory broke the model	Feeding a shorter audio clip caused the model to fail outright, rather than just being less accurate.	Kept the audio length exactly as the model was trained on, and solved the memory problem a different way (splitting the model, as above) instead.

8. Full Command Reference

For completeness / reproducibility — every command used, grouped by purpose.

8.1 Preparing an audio file for checking

```
source ~/tvm-env/bin/activate
python3 ~/prepare_audio.py /path/to/any_audio.wav
deactivate
```

8.2 Booting the secure environment (once per session)

```
cd ~/optee/build
make run QEMU_VIRTFS_ENABLE=y QEMU_VIRTFS_HOST_DIR=$HOME/qemu_share
# type: c <Enter> at the (qemu) prompt
# login: root
mkdir -p /mnt/host && mount -t 9p -o trans=virtio host /mnt/host
```

8.3 Running the check (repeat for every audio file)

```
optee_example_rawtfnet
```

8.4 Rebuilding after any code change (development only)

```
deactivate # must NOT be inside the model venv
cd ~/optee/build
make -j$(nproc)
```